

Assignment: System Design

Student Name: Raj Patil

Date: July 2026

GitHub Repository: <https://github.com/RAJ-A58/SystemDesign>

1. Project Documentation

Title Page & Overview

This document details the step-by-step implementation of three interconnected system design mini-projects built with Python, Flask, and multiprocessing. Each task solves a critical real-world infrastructure problem:

1. **Task 1 – URL Shortener Lite:** Implements custom Base62 encoding (`0-9a-zA-Z`) and in-memory hash map storage to generate unique short IDs and perform rapid HTTP 302 redirections.
 2. **Task 2 – Rate Limiter Mini:** Protects API endpoints against excessive request bursts by implementing a precise 60-second Sliding Window Log algorithm that restricts each user to 5 requests per minute (`HTTP 429 Too Many Requests`).
 3. **Task 3 – Load Balancer Simulation:** Simulates horizontal scaling and traffic distribution using a Round-Robin reverse proxy (`port 8000`) that rotates incoming requests sequentially across a cluster of 3 mock backend servers (`ports 5002, 5003, and 5004`).
-

2. Step-by-Step Implementation

Task 1: URL Shortener Lite

Process Description — Why We Made These Decisions

- **Why Base62 Encoding?** Base62 (0-9, a-z, A-Z) utilizes 62 alphanumeric characters. Unlike standard Base64, Base62 omits special characters like +, /, and =, making the short IDs 100% URL-safe and easy to type. By converting an auto-incrementing decimal counter (`self.counter = 1000, 1001...`) into Base62 (`remainder = num % 62`), we guarantee collision-free, short identifiers (`g8, g9`).
- **Why In-Memory Dictionary?** For a lightweight service without analytics requirements, an in-memory dictionary (`dict[str, str]`) provides $O(1)$ constant-time lookups for redirects, ensuring sub-millisecond response times.
- **Why HTTP 302 Found?** We return `HTTP 302 Found` instead of `HTTP 301 Moved Permanently` so that the client browser continues to hit our shortener service every time the link is clicked, preserving control over redirection logic.

Code Snippets & Explanations

1. Base62 Encoder & Storage Engine (`url_shortener/shortener.py`):

```
import string

class URLShortener:
    def __init__(self):
        # Alphabet:
        '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ' (62
        characters)
        self.characters = string.digits + string.ascii_letters
        self.base = len(self.characters) # 62
        self.url_map = {}
        self.counter = 1000 # Start at 1000 so IDs are at least 2
        characters long

    def encode(self, num):
        """Converts an integer counter into a Base62 string using
        division remainders."""
        if num == 0:
            return self.characters[0]
        encoded = []
        while num > 0:
            remainder = num % self.base
            encoded.append(self.characters[remainder])
            num //= self.base
        return ''.join(reversed(encoded))

    def shorten(self, long_url):
        """Generates unique short ID, stores the mapping, and
        increments the counter."""
```

```

        short_id = self.encode(self.counter)
        self.url_map[short_id] = long_url
        self.counter += 1
        return short_id

    def get_url(self, short_id):
        """Retrieves original URL for redirection (or returns None if
invalid)."""
        return self.url_map.get(short_id)

```

2. Flask API Endpoints (`url_shortener/api.py`):

```

from flask import Flask, request, jsonify, redirect
from shortener import URLShortener

app = Flask(__name__)
shortener = URLShortener()

@app.route("/shorten", methods=["POST"])
def shorten_url():
    data = request.get_json()
    if not data or "url" not in data:
        return jsonify({"error": "Please provide a valid 'url' inside
JSON payload."}), 400

    long_url = data["url"]
    short_id = shortener.shorten(long_url)
    short_link = f"http://127.0.0.1:5001/{short_id}"

    return jsonify({
        "message": "URL shortened successfully!",
        "short_id": short_id,
        "short_url": short_link,
        "original_url": long_url
    }), 201 # HTTP 201 Created

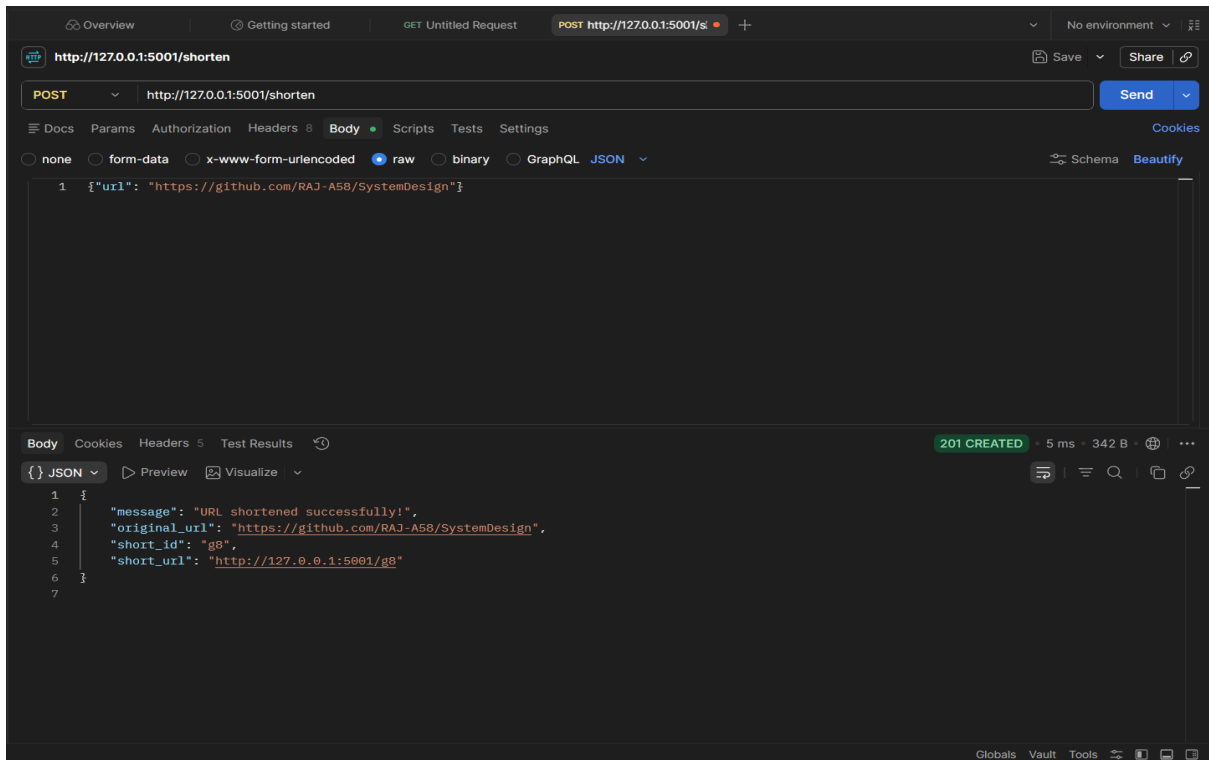
@app.route("/<short_id>", methods=["GET"])
def redirect_to_url(short_id):
    original_url = shortener.get_url(short_id)
    if original_url:
        return redirect(original_url) # HTTP 302 Found (Redirect)
    else:
        return jsonify({"error": f"Short URL ID '{short_id}' not
found!"}), 404

if __name__ == "__main__":
    app.run(port=5001, debug=True)

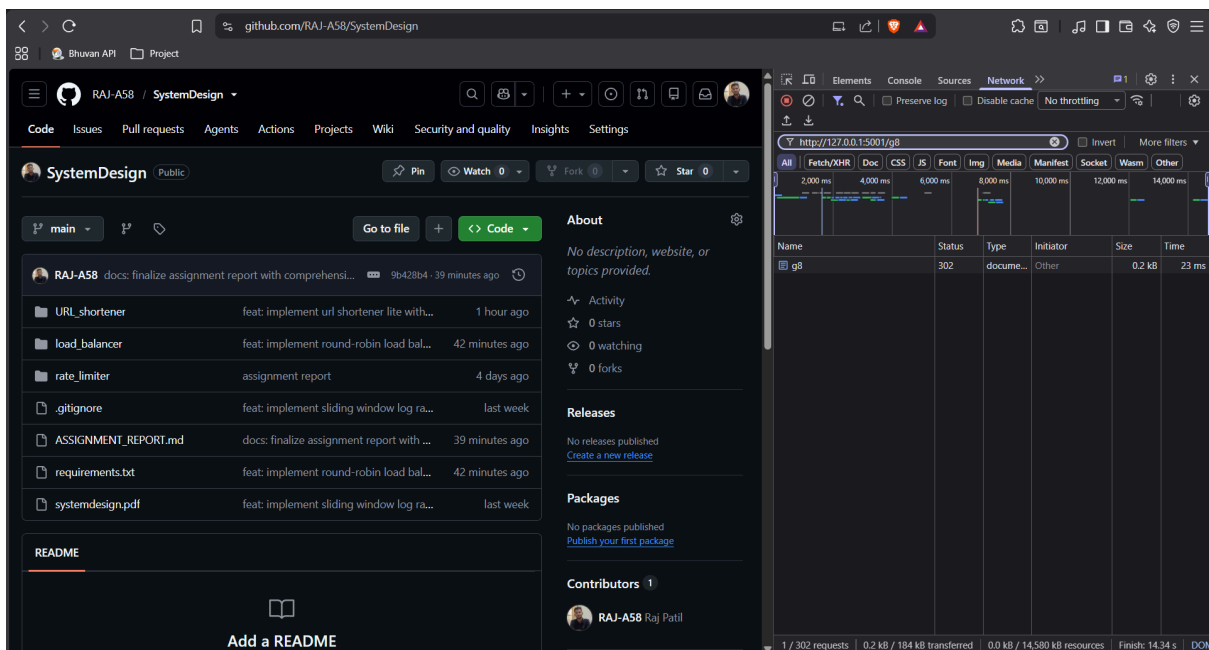
```

```
(sdev) PS E:\SystemDesign> cd .\URL_shortener\  
(sdev) PS E:\SystemDesign\URL_shortener> py api.py  
Starting URL Shortener API on http://127.0.0.1:5001  
* Serving Flask app 'api'  
* Debug mode: on  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on http://127.0.0.1:5001  
Press CTRL+C to quit  
* Restarting with stat  
Starting URL Shortener API on http://127.0.0.1:5001  
* Debugger is active!  
* Debugger PIN: 778-510-847
```

Testing API POST /shorten via Postman



Testing API Redirect GET /<shortId> via Browser



Task 2: Rate Limiter Mini

Process Description — Why We Made These Decisions

- **Why Sliding Window Log over Fixed Window Counter?** Fixed Window Counters reset at fixed intervals (e.g., exactly at the top of every minute). This introduces a severe **boundary burst flaw**: a user could send 5 requests at 12:00:59 and 5 more at 12:01:01, successfully sending **10 requests in 2 seconds**. To prevent this, we implemented the **Sliding Window Log** algorithm. It records exact request timestamps and dynamically prunes timestamps older than `time.time() - 60 seconds`.
- **Why In-Memory Dictionary (`dict[str, list[float]]`)?** Storing each `user_id` as a dictionary key pointing to a list of floats (`[1720516801.2, 1720516805.5]`) allows precise filtering and guarantees complete user isolation (Alice's requests never interfere with Bob's quota).
- **Why HTTP 429 Status Code?** Returning `HTTP 429 Too Many Requests` is the industry standard (RFC 6585) for rate limiting. It informs client applications and gateways that they have exceeded their quota and should back off before retrying.

Code Snippets & Explanations

1. Sliding Window Log Algorithm (`rate_limiter/rate_limiter.py`):

```
import time

class RateLimiter:
    def __init__(self, max_request=5, window_seconds=60):
        self.limit = max_request
        self.window_seconds = window_seconds
        self.users = {} # { "user_id": [timestamp1, timestamp2, ...] }

    def is_allowed(self, user_id):
        curr_time = time.time()
        if user_id not in self.users:
            self.users[user_id] = []

        # Step 1: Prune timestamps older than the active 60-second
        window
        window_start = curr_time - self.window_seconds
        valid_timestamps = []
        for t in self.users[user_id]:
            if t > window_start:
                valid_timestamps.append(t)
        self.users[user_id] = valid_timestamps
```

```

# Step 2: Check if current request count is under the limit (5)
if len(self.users[user_id]) < self.limit:
    self.users[user_id].append(curr_time)
    return True    # Request Allowed
else:
    return False   # Rate Limit Exceeded

```

2. Protected Flask API Endpoint (rate_limiter/api.py):

```

from flask import Flask, request, jsonify
from rate_limiter import RateLimiter

app = Flask(__name__)
limiter = RateLimiter(max_request=5, window_seconds=60)

@app.route("/data", methods=["GET"])
def get_data():
    # Identify client via ?user_id=alice (defaults to 'default_user')
    user_id = request.args.get("user_id", "default_user")

    if limiter.is_allowed(user_id):
        return jsonify({"status": "success", "message": "Data retrieved successfully."}), 200
    else:
        return jsonify({
            "error": "Too Many Requests: Try again later.",
            "message": "Too Many Requests: Try again later."
        }), 429 # HTTP 429 Too Many Requests

if __name__ == "__main__":
    app.run(port=5000, debug=True)

```

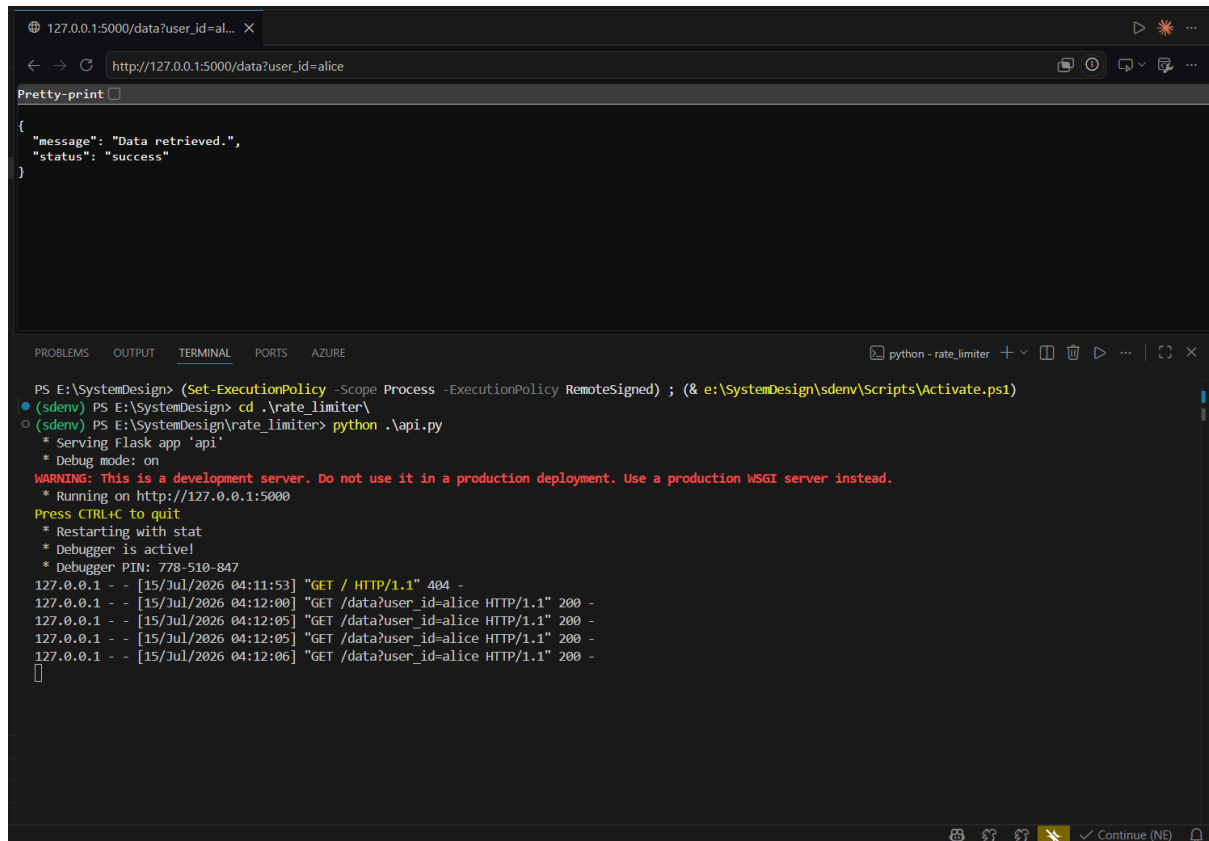
Running API Command in Terminal

```

(sdev) PS E:\SystemDesign> cd .\rate_limiter\
(sdev) PS E:\SystemDesign\rate_limiter> python .\api.py
* Serving Flask app 'api'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 778-510-847
127.0.0.1 - - [15/Jul/2026 04:06:04] "GET / HTTP/1.1" 404 -

```

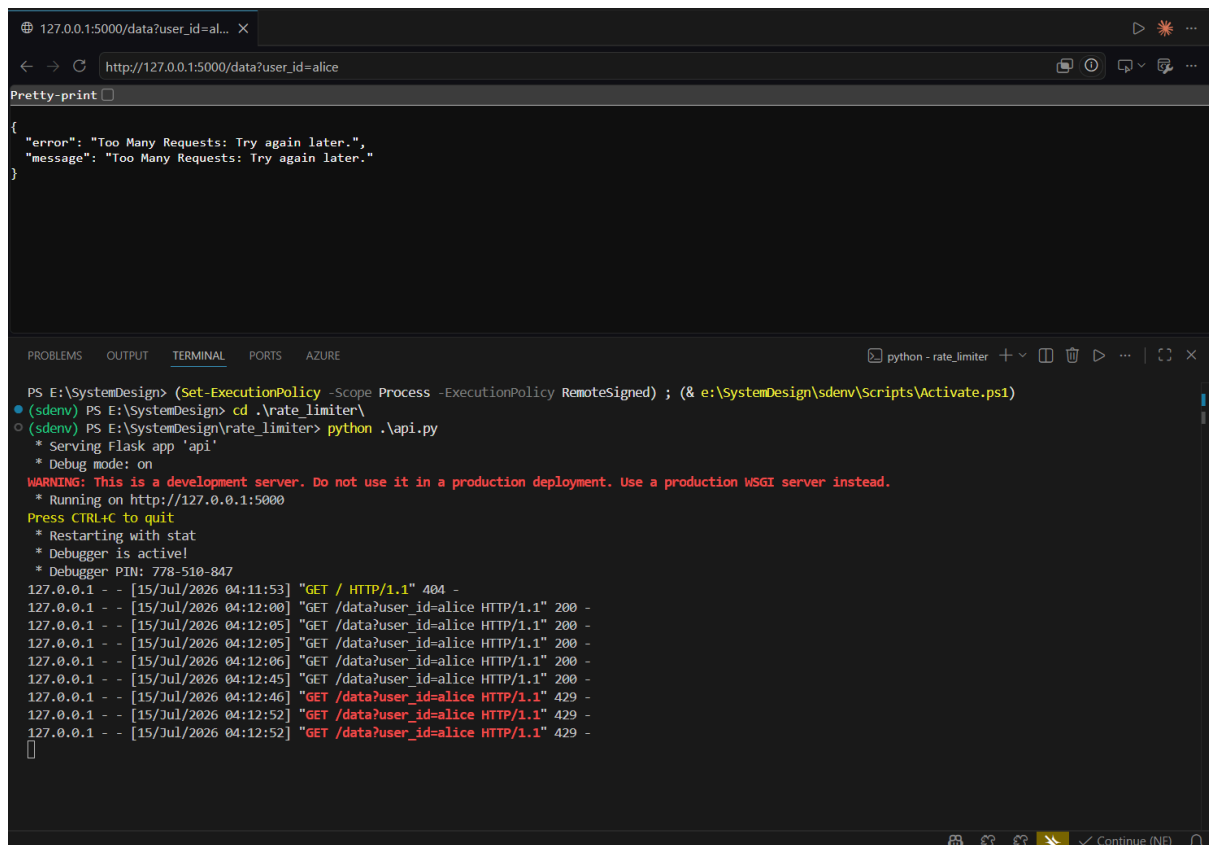
Testing API Successful Requests (1 to 5)



The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:5000/data?user_id=alice`. The page content, shown in 'Pretty-print' mode, is a JSON object: `{ "message": "Data retrieved.", "status": "success" }`. Below the browser, the VS Code terminal shows the command prompt for a PowerShell session in `E:\SystemDesign`. The user runs `(Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& e:\SystemDesign\sdev\Scripts\Activate.ps1)` and then `(sdev) PS E:\SystemDesign> cd .\rate_limiter\`. In the `(sdev) PS E:\SystemDesign\rate_limiter>` prompt, the user runs `python .\api.py`. The terminal output shows the Flask app starting in debug mode on `http://127.0.0.1:5000`. It then displays five successful GET requests from `127.0.0.1` to `/data?user_id=alice`, all returning a `200` status code.

```
PS E:\SystemDesign> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& e:\SystemDesign\sdev\Scripts\Activate.ps1)
(sdev) PS E:\SystemDesign> cd .\rate_limiter\
(sdev) PS E:\SystemDesign\rate_limiter> python .\api.py
* Serving Flask app 'api'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 778-510-847
127.0.0.1 - - [15/Jul/2026 04:11:53] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [15/Jul/2026 04:12:00] "GET /data?user_id=alice HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 04:12:05] "GET /data?user_id=alice HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 04:12:05] "GET /data?user_id=alice HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 04:12:06] "GET /data?user_id=alice HTTP/1.1" 200 -
```

Testing API Rate-Limited Request (6th Request)



The screenshot shows the same web browser window as before, but the page content is an error message: `{ "error": "Too Many Requests: Try again later.", "message": "Too Many Requests: Try again later." }`. The VS Code terminal shows the same initial setup as the previous screenshot. However, the terminal output now shows six GET requests. The first five are successful (200 status code), but the sixth request, at `127.0.0.1 - - [15/Jul/2026 04:12:46]`, returns a `429` status code, indicating a rate limit error.

```
PS E:\SystemDesign> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& e:\SystemDesign\sdev\Scripts\Activate.ps1)
(sdev) PS E:\SystemDesign> cd .\rate_limiter\
(sdev) PS E:\SystemDesign\rate_limiter> python .\api.py
* Serving Flask app 'api'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 778-510-847
127.0.0.1 - - [15/Jul/2026 04:11:53] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [15/Jul/2026 04:12:00] "GET /data?user_id=alice HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 04:12:05] "GET /data?user_id=alice HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 04:12:05] "GET /data?user_id=alice HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 04:12:06] "GET /data?user_id=alice HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 04:12:45] "GET /data?user_id=alice HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 04:12:46] "GET /data?user_id=alice HTTP/1.1" 429 -
127.0.0.1 - - [15/Jul/2026 04:12:52] "GET /data?user_id=alice HTTP/1.1" 429 -
127.0.0.1 - - [15/Jul/2026 04:12:52] "GET /data?user_id=alice HTTP/1.1" 429 -
```

Task 3: Load Balancer Simulation

Process Description — Why We Made These Decisions

- **Why Round-Robin Algorithm?** Round-Robin is the foundational algorithm for horizontal scaling. By rotating incoming traffic sequentially (`(current_index + 1) % len(servers)`), it prevents any single backend server from becoming a bottleneck and ensures uniform load distribution across the cluster.
- **Why Multiprocessing for Backend Servers?** Running multiple servers inside `multiprocessing.Process` allows us to cleanly start and terminate 3 distinct mock backend servers (Server 1 on port 5002, Server 2 on port 5003, and Server 3 on port 5004) from a single terminal script without port conflicts or thread contention.
- **Why `threading.Lock` inside Load Balancer?** Since web requests can arrive concurrently from multiple clients, wrapping our index rotation (`current_index = (current_index + 1) % 3`) in a `threading.Lock()` prevents race conditions where two threads might accidentally route to the same server simultaneously.

Code Snippets & Explanations

1. Mock Backend Cluster Launcher (`load_balancer/servers.py`):

```
from multiprocessing import Process
from flask import Flask

def create_server(server_name, port):
    """Creates mock backend server returning unique identifier string
    and port."""
    app = Flask(server_name)

    @app.route("/", methods=["GET"])
    def home():
        return f"Hello from {server_name}! (Running on port {port})\n"

    print(f"[{server_name}] Starting on http://127.0.0.1:{port}")
    app.run(host="127.0.0.1", port=port, debug=False,
            use_reloader=False)

if __name__ == "__main__":
    servers = [("Server 1", 5002), ("Server 2", 5003), ("Server 3",
5004)]
    processes = [Process(target=create_server, args=(name, port)) for
name, port in servers]
    for p in processes: p.start()
```



```
for p in processes: p.join()
```

2. Round-Robin Reverse Proxy (load_balancer/lb.py):

```
from flask import Flask, request, jsonify
import requests
from threading import Lock

app = Flask(__name__)
BACKEND_SERVERS = ["http://127.0.0.1:5002", "http://127.0.0.1:5003",
                  "http://127.0.0.1:5004"]
current_index = 0
lock = Lock()

def get_next_server():
    """Selects next server via Round-Robin modulo rotation."""
    global current_index
    with lock:
        server = BACKEND_SERVERS[current_index]
        current_index = (current_index + 1) % len(BACKEND_SERVERS)
        return server

@app.route("/", methods=["GET", "POST", "PUT", "DELETE"])
@app.route("/<path:path>", methods=["GET", "POST", "PUT", "DELETE"])
def proxy_request(path=""):
    target_server = get_next_server()
    target_url = f"{target_server}/{path}"
    try:
        resp = requests.request(
            method=request.method,
            url=target_url,
            headers={k: v for k, v in request.headers if k != 'Host'},
            data=request.get_data(),
            timeout=5
        )
        return resp.content, resp.status_code, dict(resp.headers)
    except requests.exceptions.RequestException:
        return jsonify({"error": "502 Bad Gateway", "message": f"Server {target_server} down."}), 502

if __name__ == "__main__":
    app.run(port=8000, debug=True)
```

Running Backend Servers in Terminal

```
(sdenv) PS E:\SystemDesign> cd .\load_balancer\  
(sdenv) PS E:\SystemDesign\load_balancer> python .\servers.py  
=====  
STARTING 3 MOCK BACKEND SERVERS SIMULTANEOUSLY...  
=====  
[Server 1] Starting on http://127.0.0.1:5002  
* Serving Flask app 'Server 1'  
* Debug mode: off  
[Server 2] Starting on http://127.0.0.1:5003  
* Serving Flask app 'Server 2'  
* Debug mode: off  
[Server 3] Starting on http://127.0.0.1:5004  
* Serving Flask app 'Server 3'  
* Debug mode: off  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on http://127.0.0.1:5002  
Press CTRL+C to quit  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on http://127.0.0.1:5003  
Press CTRL+C to quit  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on http://127.0.0.1:5004  
Press CTRL+C to quit
```

Running Load Balancer in Terminal

```
(sdenv) PS E:\SystemDesign> cd .\load_balancer\  
(sdenv) PS E:\SystemDesign\load_balancer> python .\lb.py  
=====  
STARTING ROUND-ROBIN LOAD BALANCER ON http://127.0.0.1:8000  
=====  
* Serving Flask app 'lb'  
* Debug mode: on  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on http://127.0.0.1:8000  
Press CTRL+C to quit  
* Restarting with stat  
=====  
STARTING ROUND-ROBIN LOAD BALANCER ON http://127.0.0.1:8000  
=====  
* Debugger is active!  
* Debugger PIN: 778-510-847
```

Load Balancing Behavior (Round-Robin Rotation)

127.0.0.1:8000 X

← → ↻ http://127.0.0.1:8000/

Hello from Server !! (Running on port 5002)

PROBLEMS OUTPUT TERMINAL PORTS AZURE

```
PS E:\SystemDesign> cd .\load_balancer\^C
PS E:\SystemDesign> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& e:\SystemDesign\sdevn\Scripts\Activate.ps1)
(sdevn) PS E:\SystemDesign> cd .\load_balancer\
(sdevn) PS E:\SystemDesign\load_balancer> python .\lb.py

=====
STARTING ROUND-ROBIN LOAD BALANCER ON http://127.0.0.1:8000
=====
* Serving Flask app 'lb'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:8000
Press CTRL+C to quit
* Restarting with stat
=====
STARTING ROUND-ROBIN LOAD BALANCER ON http://127.0.0.1:8000
=====
* Debugger is active!
* Debugger PIN: 778-510-847
[Round-Robin LB] Forwarding request to -> http://127.0.0.1:5002/
127.0.0.1 - - [15/Jul/2026 04:17:23] "GET / HTTP/1.1" 200 -
```

127.0.0.1:8000 X

← → ↻ http://127.0.0.1:8000/

Hello from Server ?! (Running on port 5003)

PROBLEMS OUTPUT TERMINAL PORTS AZURE

```
PS E:\SystemDesign> cd .\load_balancer\^C
PS E:\SystemDesign> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& e:\SystemDesign\sdevn\Scripts\Activate.ps1)
(sdevn) PS E:\SystemDesign> cd .\load_balancer\
(sdevn) PS E:\SystemDesign\load_balancer> python .\lb.py

=====
STARTING ROUND-ROBIN LOAD BALANCER ON http://127.0.0.1:8000
=====
* Serving Flask app 'lb'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:8000
Press CTRL+C to quit
* Restarting with stat
=====
STARTING ROUND-ROBIN LOAD BALANCER ON http://127.0.0.1:8000
=====
* Debugger is active!
* Debugger PIN: 778-510-847
[Round-Robin LB] Forwarding request to -> http://127.0.0.1:5002/
127.0.0.1 - - [15/Jul/2026 04:17:23] "GET / HTTP/1.1" 200 -
[Round-Robin LB] Forwarding request to -> http://127.0.0.1:5003/
127.0.0.1 - - [15/Jul/2026 04:18:33] "GET / HTTP/1.1" 200 -
```

The screenshot shows a web browser at the top with the address `http://127.0.0.1:8000/` displaying the text "Hello from Server 3! (Running on port 5004)". Below the browser is a terminal window with the following content:

```
PS E:\SystemDesign> cd .\load_balancer\VC
PS E:\SystemDesign> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& e:\SystemDesign\sdev\Scripts\Activate.ps1)
(sdev) PS E:\SystemDesign> cd .\load_balancer\
(sdev) PS E:\SystemDesign\load_balancer> python .\lb.py

=====
STARTING ROUND-ROBIN LOAD BALANCER ON http://127.0.0.1:8000
=====
* Serving Flask app 'lb'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:8000
Press CTRL+C to quit
* Restarting with stat

=====
STARTING ROUND-ROBIN LOAD BALANCER ON http://127.0.0.1:8000
=====
* Debugger is active!
* Debugger PIN: 778-510-847
[Round-Robin LB] Forwarding request to -> http://127.0.0.1:5002/
127.0.0.1 - - [15/Jul/2026 04:17:23] "GET / HTTP/1.1" 200 -
[Round-Robin LB] Forwarding request to -> http://127.0.0.1:5003/
127.0.0.1 - - [15/Jul/2026 04:18:33] "GET / HTTP/1.1" 200 -
[Round-Robin LB] Forwarding request to -> http://127.0.0.1:5004/
127.0.0.1 - - [15/Jul/2026 04:18:49] "GET / HTTP/1.1" 200 -
```

3. Final Notes

Challenges Faced & Solutions

1. **Preventing Boundary Burst Spikes in Rate Limiting:** Initially considered a simple fixed-window counter (`count++` per minute), but realized users could exploit window reset times to double their request allowance in 2 seconds. Solved this by implementing the exact **Sliding Window Log** algorithm using `time.time() - 60s` pruning.
2. **Port Management and Process Conflict:** Running multiple Flask apps concurrently inside one terminal caused port collisions and reloader loops (Address already in use). We solved this by running our 3 mock servers (5002-5004) inside `multiprocessing.Process` instances with `use_reloader=False`.
3. **HTTP Header Proxying:** When forwarding requests via `lb.py`, passing the client's `Host` header caused the backend Flask servers to reject requests. Solved by stripping `Host` from `request.headers` before forwarding with `requests.request()`.

Key Learnings

- **Core System Design Trade-offs:** Gained deep practical appreciation for why production architectures choose Sliding Window Logs or Token Buckets over simple counters.
- **Reverse Proxy Mechanics:** Built hands-on understanding of how load balancers intercept client traffic, maintain thread safety during server rotation (`(current_index + 1) % N`), and gracefully handle upstream node failures (HTTP 502 Bad Gateway).
- **RESTful API Engineering:** Mastered structuring clean Flask routes, handling JSON payloads, returning exact HTTP status codes (201 Created, 302 Redirect, 400 Bad Request, 429 Too Many Requests), and verifying behavior with Postman and terminal automation.